Comprehensive Product Requirements Document: AI-Assisted Architecture for an Aesthetic Clinic Management System

The rapid digitization of healthcare and aesthetic medicine necessitates operational frameworks that prioritize precision, patient experience, and secure data handling. For a boutique aesthetic clinic, transitioning from manual, paper-based workflows to a unified digital ecosystem is a critical evolutionary step.[1] Generic electronic medical record (EMR) systems often fail to address the highly specific needs of aesthetic practices, which rely heavily on visual documentation, package-based financial ledgers, and specialized consent workflows.[2] This document outlines the architectural, functional, and technical blueprints for a bespoke Aesthetic Clinic Management System designed to resolve these specific operational friction points.

Crucially, the development of this system will bypass traditional software engineering paradigms. Instead, the project will leverage "vibe coding"—an agent-first, AI-assisted development methodology utilizing Google Antigravity and the Model Context Protocol (MCP).[4] This approach empowers domain experts to orchestrate autonomous AI agents that plan, generate, and deploy full-stack applications directly to a Google Cloud Platform (GCP) and Firebase infrastructure.[4] The subsequent sections detail the exact specifications required to prompt the AI agents successfully, ensuring the resulting application is robust, compliant, and perfectly tailored to the clinic's workflow.

Operational Framework and User Interaction Models

To ensure the digital system seamlessly replaces and improves upon the clinic's daily operational realities, it is essential to map the distinct interaction layers of the platform. The application is designed to serve three primary functional roles, each requiring specific role-based access controls (RBAC) enforced at the database level.[7]

The first functional role belongs to the clinic administrators and front-desk personnel. These users serve as the primary entry point for patient data. Their operational responsibilities include onboarding new patients, initiating the digital registration workflows upon arrival, managing patient flow throughout the facility, and facilitating the cryptographic digital signature process during both check-in and check-out phases.[9] The interface designed for this role must prioritize rapid data entry and clear visibility into patient queues and account balances, minimizing the administrative bottleneck often experienced in high-volume aesthetic clinics.

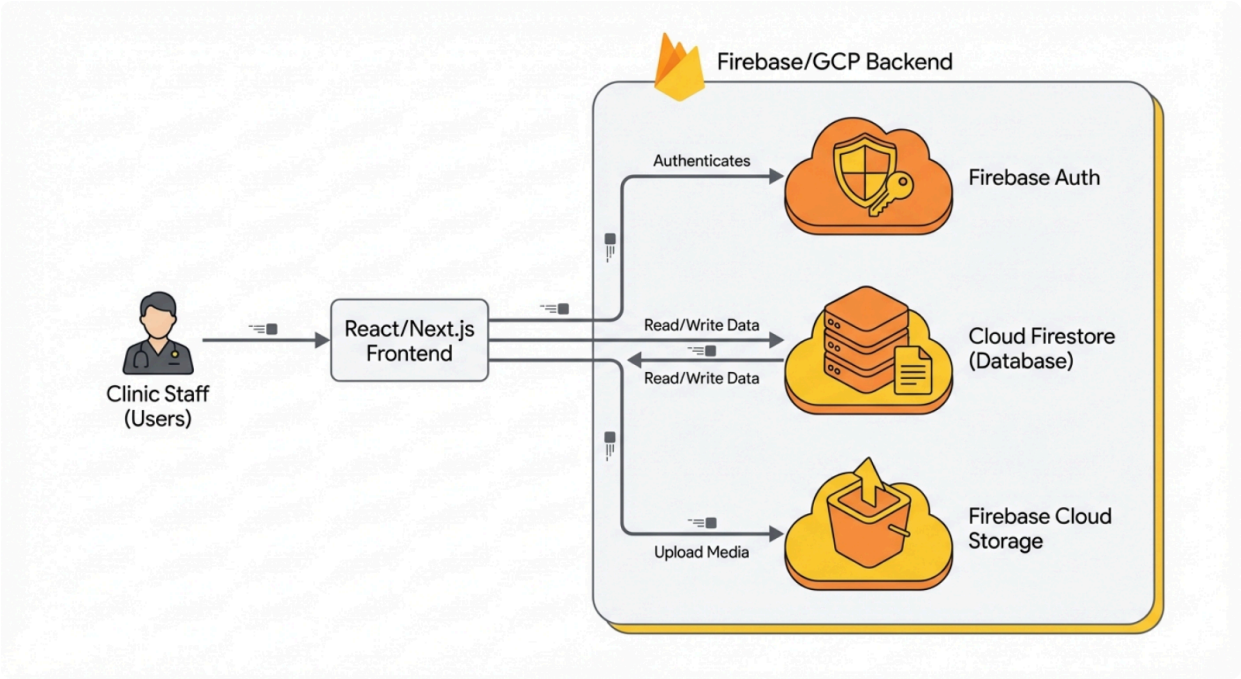
The second functional role encompasses the clinical staff, primarily nurses and medical assistants. This cohort is tasked with capturing highly specific clinical data on behalf of the

attending physician. Their workflow within the application will center around the upload of high-resolution before-and-after photographs and the logging of specific treatment parameters.[9] Because aesthetic treatments require meticulous tracking of consumable products and anatomical mapping, the interface for clinical staff must prioritize structured data entry over free-form text wherever possible, reducing the cognitive load during active patient care.[10]

The third functional role is assigned to the attending physicians and aesthetic specialists. These users rely on the system for the rapid, accurate retrieval of longitudinal patient histories. The physician interface must provide an immediate visual summary of previous treatment parameters, historical injection dosages, and comparative visual progress tracking.[3] This data retrieval is critical for informing ongoing care plans and adjusting future treatment modalities based on historical efficacy.

The convergence of these operational profiles dictates a frontend architecture optimized primarily for tablet devices, which are ubiquitous at the front desk and highly portable within treatment rooms. A responsive web interface built on a modern framework such as React or Next.js will ensure cross-device compatibility while maintaining a fluid, app-like user experience.[12]

Serverless Architecture for Aesthetic Clinic Ecosystem



Data flows securely from the clinic's frontend applications into Firebase's serverless infrastructure, utilizing Firestore for structured data and Cloud Storage for media.

Digitization of Patient Onboarding and Form Architecture

The clinic currently relies on physical registration forms that suffer from historical feature bloat and are highly susceptible to loss or damage. The transition to a digital intake process provides an opportunity to rigorously apply data minimization principles, ensuring that only clinically and administratively necessary information is captured.

A meticulous analysis of the legacy documents provided by the clinic reveals explicit instructions on which data points must be retained and which must be discarded. The legacy forms feature several sections aggressively crossed out with purple markers, indicating a deliberate move by clinic management to streamline the onboarding experience and reduce unnecessary data collection.[13]

The digital form architecture will be strictly governed by the findings detailed in the following data schema analysis.

Form Section	Original Data Field	Status for Digital System	Architectural Rationale & Implementation Notes
Client Information	Name (姓名)	Retain	Mandatory for database indexing and patient identification. Requires standard text input validation.
Client Information	Gender (性別)	Retain	Required for clinical appropriateness and demographic tracking. Render as a binary toggle or dropdown selection.
Client Information	Date of Birth (出生日期)	Retain	Essential for age verification and specific treatment contraindications. Render as a structured date picker component.
Client Information	Identity Card Number (身份證號碼)	Retain	Historically requested by the clinic. Will be rendered as an alphanumeric input. *Note: Strict compliance protocols apply regarding the collection of this data point, discussed in the*

			regulatory section.
Client Information	Occupation (職業)	**Exclude**	Explicitly crossed out on the legacy document. Deemed non-essential for clinical treatment, reducing friction during patient onboarding.[13]
Client Information	Contact Number (聯絡電話)	Retain	Primary conduit for appointment reminders and post-treatment follow-ups.[2] Requires numeric validation.
Client Information	Address (地址)	**Exclude**	Explicitly crossed out. Physical mailing addresses are obsolete for a fully digital clinic operating via SMS/Email communication.[13]
Health Condition	Taking Medication (服用藥物)	Retain	Critical clinical safety parameter. Render as a boolean (Yes/No) with conditional logic revealing a text area if "Yes" is selected.
Health Condition	Pregnant/Breastfeeding (懷孕或哺乳)	Retain	Absolute contraindication for numerous aesthetic procedures (e.g.,

			neurotoxins, specific lasers). Render as a mandatory boolean toggle.
Health Condition	Allergies (過敏史)	Retain	Crucial for preventing adverse reactions to topical anesthetics or injectables. Render as a boolean with conditional expansion for detailed text input.
Health Condition	Other Conditions (其他健康狀況)	Retain	Broad safety net for undisclosed medical issues.
Marketing Data	Source of Discovery (了解診所途徑)	**Exclude**	The entire marketing attribution section is crossed out on the legacy form. The clinic is prioritizing clinical speed over marketing analytics at the point of intake.
Consultation Details	Facial Reference Diagram (面部參考圖)	**Exclude**	The anatomical line drawings are completely crossed out. The clinic has mandated a shift away from visual drawing interfaces toward text-based logging.[9]

Consultation Details	Improvement Areas: Face, Neck	Retain	Kept on the legacy form to guide the initial consultation.
Consultation Details	Improvement Areas: Eyes, Others	**Exclude**	Crossed out on the legacy document, indicating the clinic no longer focuses on these specific categorizations during initial intake.
Consultation Details	Treatments: Injection, Laser	Retain	Primary revenue drivers and core focus of the clinic's offerings.
Consultation Details	Treatments: HIFU, Skincare	**Exclude**	Crossed out on the legacy document.
Authorization	Prior Medical Aesthetic Experience	Retain	Helps the physician gauge patient expectations and baseline tissue condition.
Authorization	Signature & Date	Retain	Legally binding confirmation of the provided medical history and consent to proceed with consultation.

The frontend implementation of this intake form must utilize dynamic rendering and conditional logic. By hiding the conditional text fields (e.g., "Please describe your allergies") until the corresponding boolean toggle is activated, the application minimizes cognitive overload for the patient.[10] Furthermore, the system must ensure that the form cannot be submitted unless all mandatory clinical safety fields are completed, enforcing absolute data completeness before the patient reaches the physician.

Cryptographic Digital Signature Management

The elimination of physical paperwork requires a robust, legally sound method for capturing patient consent. The system must implement a digital signature capture mechanism that operates flawlessly on touchscreen tablet devices positioned at the clinic's reception desk.[9]

When a patient completes the digital registration form or reaches the check-out phase, the frontend application will dynamically render an HTML5 canvas element overlaying the interface.[15] Utilizing a specialized React component library, this canvas will track user input via mouse or stylus, rendering a smooth vector representation of the signature.[17]

The technical handling of this signature data is paramount for security and storage efficiency. Upon the patient confirming the signature, the frontend application will extract the canvas data as a highly compressed Base64 encoded string or a PNG image payload.[17] Storing heavy image payloads directly within a NoSQL database document is an architectural anti-pattern that leads to severe performance degradation and excessive read costs.[19] Therefore, the system will transmit the signature payload directly to Firebase Cloud Storage via a secure API call.[21]

Firebase Cloud Storage will assign a unique identifier to the image and return a secure, authenticated URL. This URL is then injected into the patient's Firestore document payload before the final database write occurs.[22] This decoupled storage architecture ensures that the database remains lightweight and highly performant, while the actual signature images are securely archived in a dedicated storage bucket governed by strict access rules.[23] The signature is thus immutably linked to the specific timestamp of the patient's consent, providing a verifiable audit trail for the clinic's administrative records.

Clinical Remarks and the Shift to Structured Data Entry

A defining operational shift for the clinic is the abandonment of visual, diagram-based clinical charting in favor of a structured, text-based architecture.[9] Historically, aesthetic physicians relied on hand-drawing injection sites on printed facial maps. While intuitive on paper, digitizing a free-hand drawing atop a dynamic image introduces immense developmental complexity. More importantly, it results in unstructured, proprietary data formats that are exceptionally difficult to query, aggregate, or analyze computationally.[3]

To streamline the AI-assisted development process and exponentially increase the future utility of the clinic's data, the system will pivot to a highly structured "Remark" interface tailored specifically for clinical staff assisting the physician.[9]

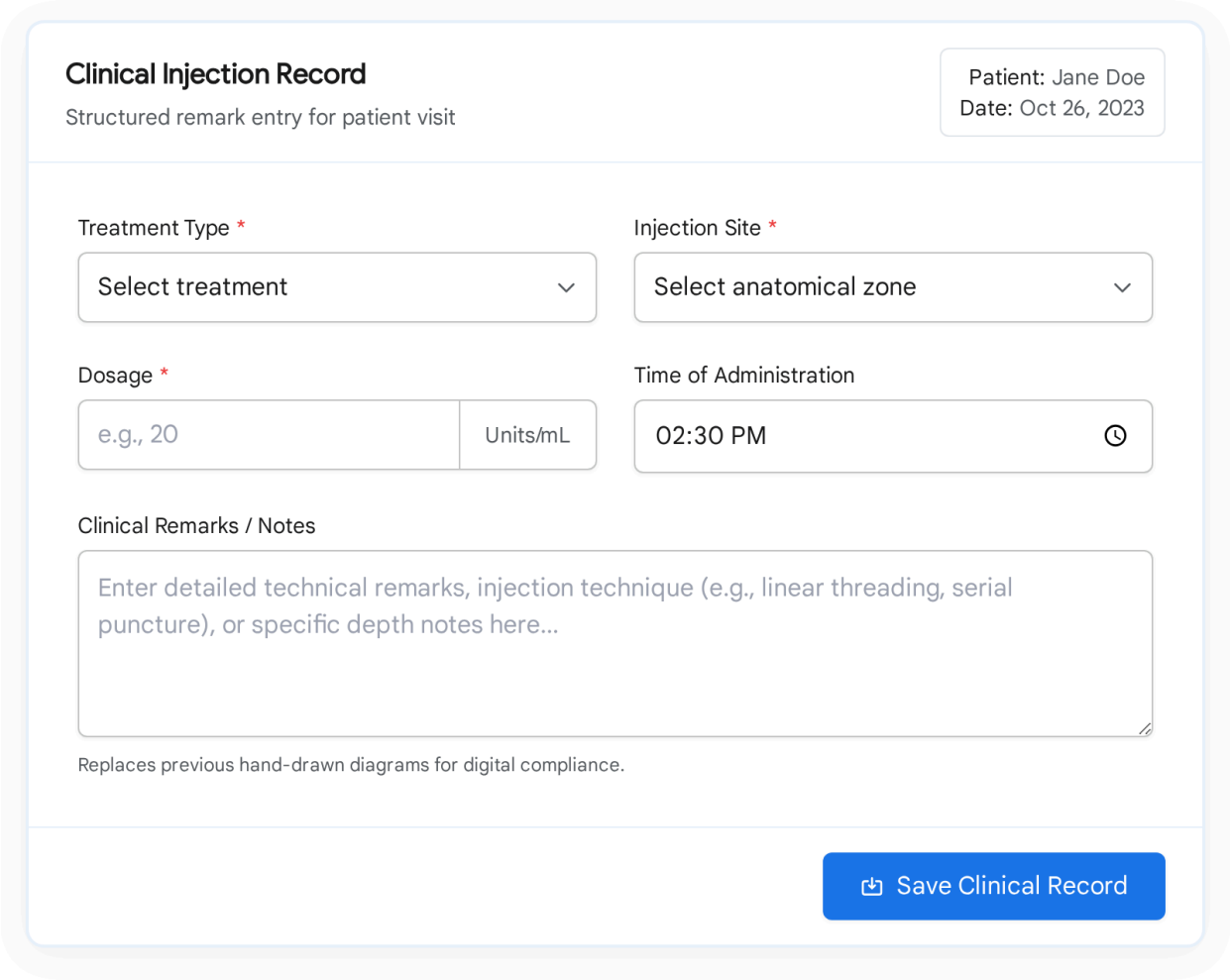
This interface will completely replace the crossed-out facial diagrams from the legacy forms. Nurses and medical assistants will utilize a targeted dashboard to log specific treatment

parameters rapidly. The required data taxonomy includes:

1. **Volumetric Dosage:** A numeric input field strictly typed to accept numerical values representing the exact volume or units of neurotoxin, dermal filler, or other injectables administered.[9]
2. **Standardized Anatomical Sites:** To prevent data fragmentation caused by varied spelling or colloquialisms (e.g., typing "frown lines" versus "glabella"), the system will utilize a predefined dropdown menu.[24] This menu will list standardized anatomical facial zones relevant to aesthetic medicine, including the Frontal region, Temporal region, Glabellar complex, Nasolabial folds, and Perioral areas.[25] This standardization ensures uniform nomenclature across the entire database, enabling precise future querying.
3. **Temporal Stamping:** An automated capture of the exact date and time the procedure was administered, crucial for calculating treatment intervals and scheduling follow-up neurotoxin or filler appointments.[9]
4. **Clinical Narrative:** A multi-line text area to capture nuanced clinical observations, patient reactions, or specific technique notes that cannot be quantified in the structured fields.

By forcing clinical data into structured JSON objects rather than static image overlays, the clinic unlocks the ability to perform complex longitudinal analytics. Administrators will eventually be able to query the database to determine aggregate product usage, calculate the average dosage per anatomical zone, or track inventory depletion rates with absolute mathematical certainty.

Optimized Clinical Data Entry Interface



By replacing complex drawing canvases with structured dropdowns and numeric inputs, the system ensures fast, queryable, and error-free clinical documentation.

Data sources: Aesthetic Medicine Operational Requirements, Eleken UI Best Practices, NCBI Medical Anatomy Guidelines

Automated Ledger and Prepaid Credit Economics

A defining characteristic of modern aesthetic clinic operations is the reliance on prepaid service packages and account credits. A primary operational vulnerability identified in the current workflow is the reliance on manual calculations by front-desk staff to determine remaining patient credit balances.[9] Manual arithmetic in a high-pressure clinical environment invariably leads to calculation errors, resulting in either revenue leakage for the clinic or trust-damaging disputes with the patient.

The application will feature an automated, mathematically rigorous ledger system designed to

entirely eliminate human calculation from the billing process.[9] The architecture of this ledger will rely on advanced database transaction protocols to ensure absolute financial integrity.

When a treatment is logged during a visit and the patient proceeds to check out, the system will not simply overwrite a single "balance" field. Doing so exposes the database to race conditions, where simultaneous network requests might corrupt the final calculation.[27] Instead, the backend process will utilize Firestore Transactions. A transaction guarantees atomicity; it reads the current balance, calculates the deduction based on the logged services, updates the master balance, and simultaneously writes an immutable receipt document to a dedicated transactions ledger.[29]

If any part of this atomic process fails or is interrupted by a concurrent update, the entire transaction is rolled back and retried, guaranteeing that the database is never left in an inconsistent state.[27] This dual-entry approach—maintaining both a current state and a verifiable history of all additions and deductions—provides a mathematically airtight history of every credit consumed.[29] Upon successful processing, the updated balance is instantly reflected on the front desk interface, allowing the patient to sign the secondary confirmation signature with full confidence in the system's accuracy.[9]

Longitudinal Visual Documentation Management

Aesthetic medicine is an inherently visual discipline. The efficacy of a treatment plan is often demonstrated not through clinical metrics, but through visual proof of transformation. The system must integrate a highly secure media management module that allows clinical staff to seamlessly upload high-resolution photographs of patients before and after specific interventions.[9]

These media assets will be transmitted directly to Firebase Cloud Storage, bypassing the primary database to maintain optimal querying performance.[21] The reference URLs generated by Cloud Storage will be embedded directly into the specific temporal visit document within Firestore, ensuring that every image is inextricably linked to the date, time, and clinical remarks of that specific session.

To maximize the impact of this visual data, the frontend application will implement a specialized React component to render the imagery. Utilizing a module such as react-before-after-slider, the interface will allow physicians and patients to overlay the pre-treatment and post-treatment images.[31] Users can drag a central vertical slider left and right to instantly and dynamically compare the baseline anatomical state with the final aesthetic outcome. This interactive visual evidence is paramount for enhancing patient satisfaction, justifying ongoing treatment plans, and fulfilling medical documentation requirements.[30]

Cloud Architecture and Firestore Schema Design

To achieve high read/write performance, robust offline capabilities, and facilitate rapid development via AI agents, Google Firebase—specifically Cloud Firestore for structured data and Cloud Storage for media—has been selected as the foundational backend infrastructure.[33]

Cloud Firestore is a schemaless NoSQL document database. Unlike traditional relational databases (SQL) that utilize rigid tables and rows, Firestore organizes data into flexible JSON-like Documents, which are grouped into Collections.[19] The performance and cost-efficiency of a NoSQL database are entirely dependent on how the data is structured. A poorly designed schema can lead to massive document sizes, slowing down retrieval times, or require excessive query reads, drastically inflating monthly cloud costs.[28]

The database schema for this clinic must be explicitly designed to minimize "read fatigue" by compartmentalizing data based on access patterns. The architecture will employ a hierarchical structure utilizing top-level collections and heavily utilizing nested subcollections to manage temporal data.[20]

Top-Level Entity Organization

The root of the database will house the primary entities.

1. **patients Collection:** This is the core directory of the application. Each document within this collection represents a unique, individual patient.[36]
    - Document ID: An auto-generated, cryptographically secure UUID provided by Firebase.
    - Root Fields: Data that is frequently accessed and rarely changes will sit at the root level. This includes firstName, lastName, phone, email, dob, gender, a nested medicalHistory object (containing the boolean flags for allergies, pregnancy, etc.), the currentCreditBalance (a numeric value constantly updated by transactions), the registrationSignatureUrl (linking to Cloud Storage), and a createdAt timestamp.[35]

Nested Temporal Subcollections

To prevent the application from downloading a patient's entire five-year history of visits every time their basic profile is opened, all chronological events will be isolated within subcollections nested beneath the specific patient document.[28] This ensures the application only queries the exact data required for the current view.

1. **visits Subcollection:** Located logically at patients/{patientId}/visits. Each time a patient enters the clinic, a new document is generated here.[20]
    - Document ID: Auto-generated UUID representing the specific session.
    - Fields: visitDate (Timestamp), an array of treatmentsPerformed (Strings representing the services rendered), and the checkOutSignatureUrl (confirming the deduction of credits and completion of the visit).[9]
2. **remarks Subcollection:** Located at patients/{patientId}/visits/{visitId}/remarks. This tightly

couples the clinical notes to the exact visit they occurred during.
    - Fields: The structured data defined earlier, including injectionSite (String), dosage (Number), time (Timestamp), and nurseNotes (String).[9]
3. **transactions Subcollection:** Located at patients/{patientId}/transactions. This serves as the immutable financial ledger for the individual.
    - Fields: amount (Number - expressed positively for top-ups and negatively for credit usage), type (String categorization such as 'purchase' or 'deduction'), timestamp (Timestamp), and a relatedVisitId reference linking the financial deduction to the clinical visit.[28]
4. **media Subcollection:** Located at patients/{patientId}/visits/{visitId}/media. This organizes the visual documentation.
    - Fields: beforePhotoUrl (String), afterPhotoUrl (String), treatmentCategory (String), and timestamp (Timestamp).[9]

This highly normalized subcollection architecture ensures that queries remain shallow and incredibly fast, regardless of how much historical data a patient accumulates over years of attendance.[36]

Security, Privacy, and Regulatory Compliance Directives

The handling of healthcare data, even within the elective aesthetic sector, carries immense regulatory liability. The system must be engineered from inception with security-by-design principles that satisfy both general cybersecurity standards and specific local privacy ordinances.

Adherence to the Hong Kong Personal Data (Privacy) Ordinance (PDPO)

The clinic operates within a jurisdiction governed by the Hong Kong Personal Data (Privacy) Ordinance. A critical compliance issue arises from the legacy forms provided by the clinic, which explicitly include a field for the patient's "Identity Card Number" (HKID).[9]

The Office of the Privacy Commissioner for Personal Data (PCPD) has issued strict guidelines regarding the beauty and aesthetic industry. According to the PCPD, the routine collection of a customer's HKID number for general registration, trial memberships, or standard beauty services is considered an excessive collection of personal data and a direct violation of Data Protection Principle 1 (DPP1).[39]

However, there is a crucial legal exception: if the aesthetic clinic employs registered medical doctors who operate energy-emitting devices or administer prescription injectables, a formal "doctor-patient relationship" is established.[39] Under these specific clinical circumstances, the

collection of the HKID number is legally permissible to ensure past medical records are correctly attributed, preventing severe medical errors.[39] Because the operational requirements explicitly mention that a "doctor sees every guest" (醫生見每一個客人) and involves injections [9], the retention of the HKID field in the digital form is justified. Nevertheless, this highly sensitive identifier must be heavily encrypted at rest within the Firestore database to satisfy DPP4 concerning data security.[41]

Furthermore, the system will enforce compliance through the following mechanisms:

1. **Consent Tracking:** The mandatory digital signatures captured during the onboarding flow explicitly satisfy the legal requirement for informed consent regarding how the patient's data will be used and processed.[43]
2. **Data Retention Automation:** To comply with DPP2 regarding the duration of retention, Firebase Cloud Functions can be engineered to automatically purge non-essential or outdated temporary records, while preserving core medical histories for the legally mandated period (typically 7-10 years).[45]

Cloud Infrastructure Security Protocols

Because the React frontend will interact directly with the Firebase backend via client-side SDKs, the database cannot rely on traditional server-side middleware for protection. Instead, the architecture must deploy rigorous Firebase Security Rules.[7]

The system will implement Firebase Authentication, enforcing email and password logins strictly limited to authorized clinic personnel.[47] The Firestore rules engine will be programmed with custom logic, such as allow read, write: if request.auth!= null;, guaranteeing that unauthenticated external traffic cannot access the patients collection.[7] Advanced rules can further restrict access based on user roles, ensuring front-desk staff cannot alter clinical remarks and nurses cannot manipulate the financial transaction ledger. Similar security rules will be applied to the Cloud Storage buckets, ensuring that highly sensitive before-and-after photographs and signature images are completely inaccessible to the public internet, preventing unauthorized scraping or data leaks.[7]

Execution Strategy: The Antigravity "Vibe Coding" Playbook

The development of this system will not utilize traditional manual coding. The project mandates the use of "vibe coding"—a modern paradigm where domain experts use natural language to direct autonomous AI agents to construct, debug, and deploy software.[4] Google Antigravity serves as the primary IDE for this endeavor.

Unlike legacy AI autocomplete tools (such as GitHub Copilot) that assist a human programmer

typing line-by-line, Google Antigravity functions as an "Agent-First" Mission Control.[4] The user assumes the role of a software architect, defining high-level goals and constraints. In response, the Antigravity AI agents autonomously generate implementation plans, manipulate the local file system, execute terminal commands to install dependencies, and even launch an automated browser to test the resulting user interface.[51]

To successfully manifest the requirements of this PRD into a production-ready application, the following highly structured vibe coding workflow must be rigorously executed within the Antigravity environment.

Phase 1: Environment Initialization and MCP Orchestration

Antigravity's capability to interact with external cloud resources is powered by the Model Context Protocol (MCP).[6] Properly configuring this environment is the absolute prerequisite to agentic development.

1. **IDE Configuration:** Launch a fresh workspace within the Antigravity IDE. Select an advanced reasoning model, specifically Gemini 3 Pro, which possesses the necessary context window and architectural understanding to scaffold full-stack applications without hallucinating complex logic.[54]
2. **Autonomy Escalation:** Navigate to the agent settings and configure the "Terminal Execution" policy to Auto. This is the cornerstone of a fluid vibe coding experience. It grants the AI agent permission to autonomously run routine infrastructure commands (e.g., npm install, creating directory structures, starting local development servers) without constantly pausing to request manual human approval.[4]
3. **Firebase MCP Server Installation:** Access the Agent Interface dropdown, open the "MCP Servers" directory, and install the Firebase MCP Server.[6] This action securely bridges the local AI environment with Google Cloud, granting the agent the necessary API access to manipulate backend services on your behalf.
4. **Natural Language Cloud Provisioning:** With the MCP active, issue a direct prompt to the agent: *"Initialize a new Firebase project for a Next.js web application. Enable Firebase Authentication (Email/Password provider), configure a Cloud Firestore Database in production mode, and set up a Cloud Storage bucket."* The Antigravity agent will interpret this command, interface with Google Cloud via the MCP, autonomously configure the backend architecture, and securely inject the resulting API keys into the local .env file.[6] This entirely eliminates the need to navigate the complex Firebase web console manually.

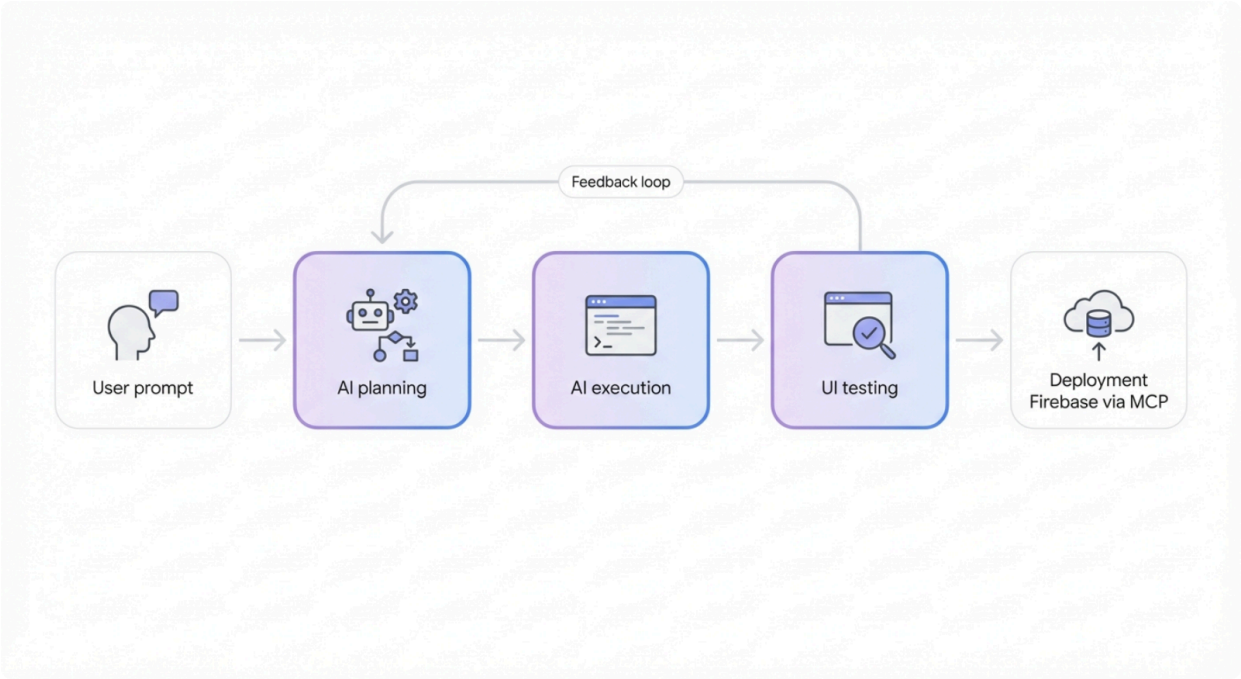
Phase 2: Agentic Planning and Core Scaffolding

A critical failure mode in vibe coding occurs when agents begin writing code without a comprehensive architectural blueprint, leading to "context rot" and convoluted codebases.[57]

The AI must be forced to plan before it executes.

1. **Defining the Vibe Profile:** Provide the agent with strict, immutable constraints by creating a GEMINI.md or .cursorrules file in the root directory.[5] This file acts as the permanent system prompt. Input the following directives: *"Tech Stack: React/Next.js, Tailwind CSS, Firebase Client SDK. UI Vibe: Clinical, minimalist, white background with subtle blue accents. Interaction Rules: Avoid complex overlapping modals; prioritize large, touch-friendly form fields optimized for tablet devices. Architecture: Strictly follow the subcollection NoSQL schema defined in the PRD."*.[5]
2. **Generating the Implementation Artifact:** Paste the core functional requirements from this document into the Antigravity Agent Panel. Instruct the agent: *"Review these requirements and generate a comprehensive implementation plan artifact. Do not write any functional code yet."*.[4] The AI will output a detailed markdown document outlining the exact component hierarchy, routing structure, and database functions it intends to build.[58]
3. **Architectural Review:** The human operator must review this generated plan to ensure the agent has accurately mapped the subcollections (visits, remarks, transactions) and understood the dual-signature workflow. Once confirmed, command the agent to proceed with execution.

The Agentic Development Lifecycle



In the Antigravity workflow, the user acts as an architect. The AI agent handles planning, code generation, terminal execution, and browser testing autonomously.

Phase 3: Vertical Slice Implementation Logic

To prevent the large language model from exceeding its context window and generating fragmented logic, the application must be built incrementally in "vertical slices." A vertical slice means constructing one single feature entirely from the database backend all the way through to the frontend UI before moving to the next feature.[59]

The sequence of prompts to the Antigravity agent should proceed as follows:

- **Slice 1: Registration and Cryptographic Signatures.** Prompt: *"Build the digital patient registration form component in React. Include fields for Name, DOB, Gender, Phone, HKID, and the specific Medical History booleans. Integrate an HTML5 canvas digital signature pad. Upon form submission, the system must first upload the canvas image to Firebase Storage, retrieve the resulting URL, and bundle it with the text data to create a new document in the 'patients' Firestore collection.".*[15]
- **Slice 2: The Immutable Visit Ledger.** Prompt: *"Develop the patient dashboard UI. Create a function to initiate a new visit. Implement a Firestore Transaction workflow: when a visit is finalized and credits are deducted, the system must atomically subtract the value from the 'currentCreditBalance' in the patient root document AND write an append-only record to*

the 'transactions' subcollection to ensure absolute financial data integrity.".[27]

- **Slice 3: Structured Clinical Remarks.** Prompt: *"Build the clinical data entry component. Replace free-text inputs with a dropdown menu for 'Injection Site' (populated with standard facial zones) and a numeric input for 'Dosage'. Configure this component to write data specifically to the 'remarks' subcollection nested beneath the active visit document.".*[9]
- **Slice 4: Visual Progress Tracking.** Prompt: *"Implement a secure image upload component for the clinical staff to upload 'Before' and 'After' treatment photographs to Firebase Storage. Build a dedicated UI view utilizing a React image comparison slider component, allowing the physician to dynamically reveal the before and after images by dragging a central divider.".*[30]

Phase 4: Autonomous Empirical Testing and Deployment

A profound capability of the Antigravity environment is its ability to test its own code empirically using an integrated Browser Sub-Agent.[4]

1. **Automated QA:** After a vertical slice is generated, issue the command: *"Launch the local development server. Utilize the browser agent to navigate to the registration form. Fill the form with dummy patient data, interact with the signature canvas, submit the form, and verify that the data successfully appears within the Firebase Firestore console."* The agent will execute this task, capturing screenshots and screen recordings of its interactions, providing visual proof to the human operator that the code functions perfectly in a real browser environment.[4]
2. **Frictionless Deployment:** Once all vertical slices are complete and verified locally, the system is ready for the internet. Leverage the Firebase MCP integration for a final command: *"Deploy the compiled Next.js application to Firebase Hosting."* The Antigravity agent will autonomously manage the build process, configure the necessary hosting parameters, and execute the deployment scripts, returning a live, production-ready URL.[6]

By strictly adhering to these structured data schemas, regulatory compliance frameworks, and the specialized vibe coding orchestration strategy, domain experts can rapidly manifest a highly functional, bespoke aesthetic clinic management system without requiring traditional software engineering expertise.

Works cited

1. Aesthetic Clinic Management Softwares – Can We Improve Patient, accessed March 9, 2026, https://www.researchgate.net/publication/375078231_Aesthetic_Clinic_Management_Softwares_-_Can_We_Improve_Patient_Safety
2. The Practice Management Software Aesthetic Medics Actually Need, accessed March 9, 2026,

https://facesconsent.com/blog/the-pms-aesthetic-medics-actually-need/
3. Before & After Photo & Video Management - Aesthetic Record EMR, accessed March 9, 2026, https://www.aestheticrecord.com/before-after-photo-video-managment/
4. Vibe Coding Explained: Tools and Guides - Google Cloud, accessed March 9, 2026, https://cloud.google.com/discover/what-is-vibe-coding
5. Google Antigravity & Vibe Coding: Gemini 3 Pro Developer Guide, accessed March 9, 2026, https://vertu.com/ai-tools/google-antigravity-vibe-coding-gemini-3-pro-developer-guide-claude-code-comparison/
6. AntiGravity Firebase Integration: Build Apps With One Command, accessed March 9, 2026, https://www.reddit.com/r/AISEOInsider/comments/1qnifd3/antigravity_firebase_integration_build_apps_with/
7. Get started with Cloud Firestore Security Rules - Firebase - Google, accessed March 9, 2026, https://firebase.google.com/docs/firestore/security/get-started
8. Fine-grained Access Control for an Online Medical Records Portal, accessed March 9, 2026, https://medium.com/@divineoffishal/fine-grained-access-control-for-an-online-medical-records-portal-with-next-js-c535594123a9
9. Aesthmetic Medicine Website Design Background
10. Healthcare UI Design 2026: Best Practices + Examples - Eleken, accessed March 9, 2026, https://www.eleken.co/blog-posts/user-interface-design-for-healthcare-applications
11. 6 Best Aesthetic EMR Software for 2026 | Expert-Reviewed - Pabau, accessed March 9, 2026, https://pabau.com/blog/aesthetic-emr-software/
12. Google Antigravity Tutorial: Build a Finance Risk Dashboard, accessed March 9, 2026, https://www.datacamp.com/tutorial/google-antigravity-tutorial
13. accessed March 9, 2026, uploaded:Form1.jpg-d1c9e8d7-0a3c-46a5-bab2-693712032e01
14. Aesthetic Surgery Clinic Intake Form Template - Jotform, accessed March 9, 2026, https://www.jotform.com/form-templates/aesthetic-surgery-clinic-record
15. How To Build a Digital Signature Pad in React - Common Ninja, accessed March 9, 2026, https://www.commoninja.com/blog/how-to-build-a-digital-signature-pad-in-react
16. Create simple Signature pad with HTML5 Canvas and jQuery Mobile, accessed March 9, 2026, https://uithought.wordpress.com/2014/01/18/create-simple-signature-pad-with-html5-canvas-and-jquery-mobile/
17. React Signature Pad Component | Syncfusion, accessed March 9, 2026, https://www.syncfusion.com/react-components/react-signature
18. Implementing a signature pad with JavaScript - LogRocket Blog, accessed March 9, 2026, https://blog.logrocket.com/implementing-signature-pad-javascript/

19. Choose a data structure | Firestore - Firebase - Google, accessed March 9, 2026, https://firebase.google.com/docs/firestore/manage-data/structure-data
20. How to design a Cloud Firestore database schema - Stack Overflow, accessed March 9, 2026, https://stackoverflow.com/questions/47076373/how-to-design-a-cloud-firestore-database-schema
21. Get started with Cloud Storage on web - Firebase, accessed March 9, 2026, https://firebase.google.com/docs/storage/web/start
22. Signature | FlutterFlow Documentation, accessed March 9, 2026, https://docs.flutterflow.io/resources/ui/widgets/built-in-widgets/signature/
23. Firebase Storage for Web: Uploading & Downloading An ... - YouTube, accessed March 9, 2026, https://www.youtube.com/watch?v=31MVlwvstzs
24. Dropdown Menu UI: Best Practices and Real-World Examples - Eleken, accessed March 9, 2026, https://www.eleken.co/blog-posts/dropdown-menu-ui
25. Facial fillers: Relevant anatomy, injection techniques, and ... - PMC, accessed March 9, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10541169/
26. Facial topography of the injection areas for dermal fillers, and ... - BVS, accessed March 9, 2026, https://docs.bvsalud.org/biblioref/2016/10/2115/2013_234_ingles.pdf
27. Firestore - Transactions and batched writes - air native extensions, accessed March 9, 2026, https://docs.airnativeextensions.com/docs/firebase/firestore/transactions-and-batched-writes/
28. Firestore schema to store users' weekly/monthly stats : r/Firebase, accessed March 9, 2026, https://www.reddit.com/r/Firebase/comments/w2asqt/firestore_schema_to_store_users_weeklymonthly/
29. Transactions design in Firebase with Firestore - Stack Overflow, accessed March 9, 2026, https://stackoverflow.com/questions/73239119/transactions-design-in-firebase-with-firestore
30. Before and After Photo Creation in Your EMR - emilyEMR, accessed March 9, 2026, https://emilyemr.ai/before-and-after-photo-creation-in-your-aesthetic-emr/
31. smeleshkin/react-before-after-slider-component - GitHub, accessed March 9, 2026, https://github.com/smeleshkin/react-before-after-slider-component
32. Managing Before and After Photos in Aesthetic Practice, accessed March 9, 2026, https://www.aestheticnursesoftware.com/managing-before-and-after-photos
33. Firestore | Google Cloud, accessed March 9, 2026, https://cloud.google.com/products/firestore
34. Cloud Firestore - React Native Firebase, accessed March 9, 2026, https://rnfirebase.io/firestore/usage
35. Cloud Firestore Data model | Firebase - Google, accessed March 9, 2026, https://firebase.google.com/docs/firestore/data-model
36. Achieving the Perfect Firestore Schema (Case Study) | by Asfar Ali, accessed

March 9, 2026, https://medium.com/@s4.ali/how-to-achieve-a-perfect-firestore-schema-case-study-de6af05bfcbb
37. Firestore pricing | Google Cloud, accessed March 9, 2026, https://cloud.google.com/firestore/pricing
38. Firestore: Version history of documents - Stack Overflow, accessed March 9, 2026, https://stackoverflow.com/questions/53442460/firestore-version-history-of-documents
39. Proper Handling of Customers' Personal Data for the Beauty Industry, accessed March 9, 2026, https://www.pcpd.org.hk/english/resources_centre/publications/files/BeautyIndustry_ENG.pdf
40. Proper Handling of Customers' Personal Data for the Beauty Industry, accessed March 9, 2026, https://www.pcpd.org.hk/english/resources_centre/publications/posters_Infographics/files/pcpd_beauty_industry.pdf
41. The Personal Data (Privacy) Ordinance - PCPD, accessed March 9, 2026, https://www.pcpd.org.hk/english/data_privacy_law/ordinance_at_a_Glance/ordinance.html
42. Privacy Protection & Data Security in Digital Healthcare Environment, accessed March 9, 2026, https://www.ehealth.gov.hk/filemanager/content/pdf/common/training/2022/10/14/privacy-protection-and-data-security-in-digital-healthcare-environment.pdf
43. Legal requirements for patient data in the aesthetic business, accessed March 9, 2026, https://meridiq.com/en/security/legal-requirements-for-patient-data-management-in-the-aesthetic-business/
44. What to Consider Regarding Before-and-After Photos in Med Spas, accessed March 9, 2026, https://www.littlehealthlaw.com/blog/what-to-consider-regarding-before-and-after-photos-in-med-spas/
45. Medical record keeping - a guide for healthcare professionals, accessed March 9, 2026, https://www.medicalprotection.org/hongkong/guidance/medical-record-keeping
46. Usage and limits | Firestore - Firebase, accessed March 9, 2026, https://firebase.google.com/docs/firestore/quotas
47. How to Integrate React with Firebase for Authentication and, accessed March 9, 2026, https://www.cybrosys.com/blog/how-to-integrate-react-with-firebase-for-authentication-and-database-storage
48. Adding Authentication with Firebase to Your Web App, accessed March 9, 2026, https://www.jobaajlearnings.com/blog/adding-authentication-with-firebase-to-your-web-app
49. dhruvpatel16120/clinic-management-system - GitHub, accessed March 9, 2026,

https://github.com/dhruvpatel16120/clinic-management-system
50. Tutorial : Getting Started with Google Antigravity - Medium, accessed March 9, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-google-antigravity-b5cc74c103c2
51. Google Antigravity | Vibe Coding Tool Review & Features, accessed March 9, 2026, https://www.vibe-coding.uk/tools/google-antigravity
52. Cursor vs Google AntiGravity : Best AI IDE for Vibe Coders? - Medium, accessed March 9, 2026, https://medium.com/data-science-in-your-pocket/cursor-vs-google-antigravity-best-ai-ide-for-vibe-coders-f3ecbd02f161
53. Antigravity Editor: MCP Integration, accessed March 9, 2026, https://antigravity.google/docs/mcp
54. I tried Google's new Antigravity IDE so you don't have to (vs Cursor, accessed March 9, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1p35bdl/i_tried_googles_new_antigravity_ide_so_you_dont/
55. Weightless Code: My 7-Day Experiment with Google Antigravity, accessed March 9, 2026, https://dev.to/naresh_007/weightless-code-my-7-day-experiment-with-google-antigravity-9g5
56. AntiGravity Firebase Integration: The Future of No-Code App, accessed March 9, 2026, https://juliangoldie.com/antigravity-firebase-integration/
57. New to Antigravity: seeking workflow advice for full-stack apps, accessed March 9, 2026, https://www.reddit.com/r/google_antigravity/comments/1qxwfd4/new_to_antigravity_seeking_workflow_advice_for/
58. Getting Started with Google Antigravity, accessed March 9, 2026, https://codelabs.developers.google.com/getting-started-google-antigravity
59. A Structured Workflow for "Vibe Coding" Full-Stack Apps - Dev.to, accessed March 9, 2026, https://dev.to/wasp/a-structured-workflow-for-vibe-coding-full-stack-apps-352l
60. How to create a React signature pad - Nutrient, accessed March 9, 2026, https://www.nutrient.io/blog/how-to-create-a-react-js-signature-pad/
61. Antigravity and Firebase MCP accelerate app development, accessed March 9, 2026, https://firebase.blog/posts/2025/11/firebase-mcp-and-antigravity/